CMSC 14300: Systems Programming I

The University of Chicago, Spring 2024

Adam Shaw, Matthew Wachs

Welcome!

In CMSC 14300, students, building on their experience with applications programming and fundamental data structures in Python, develop a deeper understanding of a computer's operations when executing a program. In order to make the inner workings of the computer more transparent, we study the C programming language, with special attention devoted to bit-level programming, pointers, allocation, the stack/heap distinction, memory management, and file input and output. C is the *lingua franca* of computer programmers, and, though not without flaws, it remains one of the great historic successes in the design of practical programming languages.

Unlike Python, where typing is available but unenforced, C mandates that programs are correctly typed in order to compile them. We consider static typing and the role it plays in program development and memory layout. We revisit basic data structures including strings, arrays, lists, trees, and dictionaries (hash tables) and view them in closer detail, and from a slightly different perspective. Furthermore, we begin an ongoing examination of how memory is organized and structured in a modern computer.

Students in CMSC 14300 will

- acquire a wide-ranging C programming toolkit, including control structures, functions, enumerations, and struct and union definitions,
- use static typing to guide program development,
- employ recursion and iteration as complementary methods for repetitive computation,
- understand binary representations of data, and write programs operating at the level of bits and bytes,
- program with pointers, understand pointer arithmetic, and learn how to use indirection operators,
- learn to allocate memory directly and detect and correct memory leaks,
- weigh the advantages and disadvantages of linked and contiguous data structures as alternatives to one another,
- build dictionaries (hash tables) from scratch, and

- practice basic and advanced debugging techniques with `lldb` and `valgrind`.

Having learned a second programming language well, and consequently having viewed a variety of common problems from distinct vantage points, students who complete CS143 will begin to see past the superficial characteristics of particular computer programs to appreciate their deeper properties.

Instructors

Adam Shaw, email: `ams@cs.uchicago.edu`, office: Crerar 213.

Matthew Wachs, email: `mwachs@cs.uchicago.edu`. office: Crerar 211.

Course Coordinator

Víctor Almarez Argueta, email: `jesusaa@uchicago.edu`

Contacting Us The communications framework for this course will consist of a combination of online systems: Canvas, Gradescope, and Ed Discussion. It can be a bit confusing to negotiate so many different systems, but to try to keep it simple, we will favor Ed Discussion as the primary point of contact, especially for questions and answers about the course content. Canvas will contain office hour schedules, homework assignments, and other documents, and Gradescope will be used for grading and managing regrade requests.

Using Ed Discussion Ed Discussion is where most of our communications will take place. In cases where you have a question that is about your own personal situation and not relevant to the class as a whole, please ask a "private question" on Ed; your classmates will not be able to see any private questions. You should know that anonymous Ed Discussion posts are anonymous as far as other students are concerned, but, even in anonymous posts, identities are still visible to the staff.

In CS143, we will not look into repositories in response to Ed questions. In practical terms, this means the phrase "my code is pushed" should not be a standard part of an Ed post. We may ask you to share some code with us in a private Ed post, but we will not on our own initiative inspect your code in its repository and tell you what's wrong with it. The rationale for this policy is to help you build self-reliance and encourage you to improve your ability to identify problems in your own code and ask pointed questions about what problems it might contain.

Lectures

- MWF 9:30–10:20, Wachs, Stuart 102 (section 1)
- MWF 10:30–11:20, Wachs, Stuart 102 (section 2)
- MWF 11:30–12:20, Shaw, Ryerson 251(section 3)

- MWF 2:30–3:20, Shaw, Stuart 105 (section 4)

The first lecture is on Monday, March 18; the last is on Friday, May 17.

Electronic Devices We do not allow the use of electronic devices during lectures. They are too distracting and create a barrier between instructor and student. This includes laptops, smartphones, and tablets. The lone exception to this policy is for students whose handwriting issues necessitate their use of a device for note taking, who will be permitted to use a plain text editor or notepad on a device whose wireless capability is turned off. If you are such a student, please talk to your instructor.

Lab Sessions Students must register for and attend lab sessions each week. Lab sessions are held in the Computer Science Instructional Laboratory (also known as the CSIL); it is located on the first floor of Crerar Library. Attendance at the lab session for which you are registered is mandatory under the honor system; your work in labs will not be graded, although content learned in labs may appear on the quiz or the exam.

If labs are ungraded, *i.e.*, they "don't matter," why bother to attend them? The labs are high-quality specially-designed exercises that complement our curriculum and are designed just for you. Go to labs because you want to learn as much as you can in CS143 and you want to become the best, most capable programmer you can and move closer to reaching your full potential.

The lab times are as follows:

Tues 2:00pm–3:20pm, 3:30pm–4:50pm, 5:00pm–6:20pm, 6:30pm–7:50pm

Schedule of Topics by Week (subject to change)

Week	**Topics**
1	basics: the terminal, `printf`, types, functions, conditionals
2	operators, bits, binary encoding
3	pointers, allocation, arrays
4	strings, structs, unions
5	linked lists
6	sorting
7	pthreads
8	trees
9	hash tables

Office Hours To be announced once the quarter starts.

Texts (required) *The C Programming Language (Second Edition)*, Kernighan,

Ritchie. The textbook is available on campus at the Seminary Co-op Bookstore[1]; you can of course find new and used copies at online bookstores as well.

Software All the software we use in this course is available free of charge for all common platforms. We will mainly use *vim* for editing, *clang* for compilation, and *git* for coursework management. In addition, we will practice debugging and memory-use analysis with *lldb* and *valgrind*.

Please note that while you can install most or all of this software on your own computer, it is always possible for any student to use the department's software using *ssh*, which is to say that it is not strictly necessary to install any software if one is inclined to work over a network connection. We will discuss and demonstrate this method of working early in the quarter.

Grading Coursework is comprised of homework assignments, the quiz, and the exam. There are also weekly lab exercises, but those are not graded.

Each student's final grade will be computed according to the following formula: quiz 10%, final exam 25%, and homework 65%. We will scale the grades, so what precisely constitutes an A, B, *etc.* will be determined by the collective performance of the class. We announce in advance that a 92 guarantees at least an A- for the course; an 82 guarantees a B-; a 72 guarantees a C-; a 65 guarantees a D. The scale may shift downward from this somewhat, meaning that the eventual grading boundaries may be more inclusive than this, but not less so.

Homework There will be weekly homework assignments. (There will be no homework assignment during exam week.) These will generally be due on Mondays at 9:00pm.

Quiz and Exam There will be an in-class quiz around week 4 and a late midterm exam in the evening on Monday, May 6.

The quiz is planned for Wednesday, April 10, during regular class time. We may decide to change this date; please stay tuned.

The exam on May 6 is from 7pm–9pm in Kent 107. Please make sure to be available at that time. The exam is technically not a final exam but a late midterm; it will cover most but not all of the course content. To be clear, there will be no CS143 exam during finals week at the very end of the quarter.

Special accommodations for exams should be arranged through SDS. You can contact SDS at any time in advance of the quiz and/or the exam; the sooner you reach out, the better.

Regrade Requests Graders, TAs, and instructors do make mistakes, so it is normal and expected that a few assignments will have grading errors. In

[1] 5751 S. Woodlawn Avenue; `https://www.semcoop.com`.

order to manage the workload of regrade requests, however, regrade requests must be submitted on Gradescope within five weekdays of the assignment being returned.

Late Work Deadlines in this course are rigid. Since you submit your work electronically, deadlines are enforced to the minute. Late work will not be counted. We will accept late work in the case of special circumstances, when those circumstances are extraordinary (being busy doesn't count).

Artificial Intelligence Students are prohibited from using AI tools such as ChatGPT as any part of their CS143 coursework. In future quarters, as AI tools weave their way into modern life and we develop a more subtle and sophisticated collective understanding of them, we may adopt a more nuanced policy. But, for now, this quarter, AI is banned. We encourage you to learn to write code without the support of robotic partners. Use of generative AI to do your homework or help with your homework is considered a violation of academic honesty policy.

Academic Honesty In this course, as in all your courses, you must adhere to college-wide honesty guidelines as set forth at `https://college.uchicago.edu/policies-regulations/academic-integrity-student-conduct`. The college's rules have the final say in all cases. Our own paraphrase is as follows:

1. Never copy work from any other source and submit it as your own.
2. Never allow your work to be copied or seen by another student.
3. Never submit work identical to another student's.
4. Never look at someone's working solution in order to solve a problem.
5. Document all collaboration.
6. Cite your sources.

We are serious about enforcing academic honesty. If you break any of these rules, you will face tough consequences. Specifically, any student who is determined to have participated in academic dishonesty will not be allowed to withdraw or take the course pass/fail, and will receive a course grade no higher than a C.

Please note that sharing your work publicly (such as posting it to the web) definitely breaks the second rule. With respect to the third rule, you may discuss the general strategy of how to solve a particular problem with another student (in which case, you must document it per the fifth rule), but you may not share your work directly, and when it comes time to sit down and start typing, you must do the work by yourself. If you ever have any questions or concerns about honesty issues, raise them with your instructor, early.

Accessibility Statement The University of Chicago is committed to ensuring equitable access to our academic programs and services. Students with disabilities who have been approved for the use of academic accommodations by Student Disability Services (SDS) and need a reasonable accommodation(s) to participate fully in this course should follow the procedures established by SDS for using accommodations. Timely notifications are required in order to ensure that your accommodations can be implemented. Please contact Víctor Almaraz Argueta, the Course Coordinator, to discuss your access needs in this class after you have completed the SDS procedures for requesting accommodations.

SDS Phone: (773) 702-6000

SDS Email: disabilities@uchicago.edu

Course Coordinator: jesusaa@uchicago.edu (Víctor Almarez)

Advice Writing code that does what it is supposed to do can be pleasant, even joyful. By contrast, fighting for hours with broken code is the opposite. We would like to help you experience more of the former and less of the latter. Work methodically. Start your work well ahead of time. Beyond a certain point, it is not profitable to be stumped. If you have made no progress in some nontrivial chunk of time, say, an hour or two, it is time to stop coding and change your approach (or perhaps switch contexts and do something else for a while). Use one of our many support mechanisms to get some assistance (office hours, Ed Discussion). In summary, be kind to yourself and don't allow yourself to grind away without any forward motion. We will help you get going again when you are stuck.